# PWM-Dyn and ParadoxBench: A Provenance-Audited Physical Dynamics Model, Benchmarked Against Zero-Shot Foundation Models on US Traffic and Indian Rainfall

Aditya Kumar Anupam Singh

Paradox AI Lab, India

paradoxailab.com

October 2026

**Abstract**

We present PWM-Dyn, the first trained component of the Paradox World Model (PWM): a 0.3M-parameter spatiotemporal dynamics model that learns how physical conditions evolve and propagate over real-world networks — road-sensor graphs and geographic rainfall grids — with calibrated quantile uncertainty and robustness to missing sensors. PWM-Dyn is trained from random initialisation on real observations only: Caltrans PeMS highway traffic (PEMS08, PEMS04) and India Meteorological Department 0.25° gridded daily rainfall. We benchmark it head-to-head against Google's TimesFM 2.5 and TimesFM 3.0 zero-shot time-series foundation models on identical held-out test windows, scored by one shared evaluation function, with the opponents isolated in a separate software environment. A provenance audit proves every PWM checkpoint contains only PWM-Dyn's own parameters and that PWM's code imports no external model library. Measured results: on PEMS08 (1-hour horizon), PWM-Dyn's 3-seed ensemble reaches test MAE 14.18 — 3rd against the published leaderboard, ahead of STID and Graph WaveNet — and, on identical test windows, beats zero-shot TimesFM 2.5 (MAE 15.04) by 5.7%, TimesFM 3.0 univariate (14.63) by 3.1% and TimesFM 3.0 multivariate (14.55) by 2.6%. On PEMS04 it reaches MAE 18.50 (4th on the published table), 6.1% below TimesFM 2.5 (19.70) and 3.3% below TimesFM 3.0 multivariate (19.14). On Indian monsoon-belt rainfall it reaches MAE 2.85, beating climatology by 20%, persistence by 35%, TimesFM 2.5 (2.96) by 3.9% and TimesFM 3.0 multivariate (2.92) by 2.5% — while both TimesFM models are better calibrated there (84–86% interval coverage vs PWM's 55%), a documented defect of PWM on rainfall extremes. PWM-Dyn thus has lower point error than every opponent round on all three datasets (7 of 7). We situate PWM-Dyn against the published PEMS08 leaderboard and against the global landscape of physical-world models from US, Chinese and European labs, and we state explicitly which comparisons are measured, which are quoted from authors, and which are not meaningful. Our claim is deliberately narrow: a small regional specialist with verifiable provenance can be competitive with — and in places exceed — large zero-shot foundation models on physical forecasting tasks, at a fraction of the parameters and training cost.

## 1 Introduction

Physical world models — systems that learn how the physical world is structured, how it changes, and what happens next — are advancing along several distinct fronts: generative interactive environments (DeepMind Genie 3, World Labs Marble), physical AI for robotics (NVIDIA Cosmos), video-based physical reasoning (Meta V-JEPA 2), global weather (Google WeatherNext/GraphCast, Huawei Pangu-Weather, Shanghai AI Lab FengWu, Fudan FuXi), and measured-state forecasting over sensor networks. These fronts share an ambition but not a benchmark: no single score compares a video world model with a weather model or a traffic model, and any claim of beating "all world models" is unfalsifiable. This paper takes the opposite approach: pick tasks where exact, like-for-like comparison is possible, and make every provenance and fairness choice auditable.

We ask a concrete question: **on real physical forecasting tasks, how does a small specialist trained from scratch on a region compare with a frontier zero-shot time-series foundation model?** The question matters for teams outside hyperscale labs — if a 0.3M-parameter regional model with honest uncertainty can match or beat a 200M–1B-parameter zero-shot foundation model on the region's own physics, then capable physical-world intelligence is buildable at startup scale, domain by domain.

Contributions:

1. **PWM-Dyn**, a compact graph-based dynamics model (temporal encoder + physical-graph diffusion + periodic embeddings + quantile heads) that trains in minutes-to-hours on one GPU, handles offline sensors by construction (mask-aware training), and reports calibrated 80% prediction intervals.
2. **A provenance-audited benchmark harness**: opponents run zero-shot in a separate software environment; a static+binary audit proves PWM's checkpoints contain only PWM's own parameters and PWM's code imports no external model library. "Not a wrapper" is verified, not asserted.
3. **Measured head-to-head results** against Google TimesFM 2.5 (Apache-2.0) and TimesFM 3.0 (univariate and multivariate) on US highway traffic (PEMS08, PEMS04) and Indian monsoon-belt rainfall (IMD 0.25° central India, 2000–2024), under rules that favour the foundation model (it receives 17–42× longer input history than PWM).
4. **An honest placement** of the results in the global landscape, separating measured comparisons, author-published numbers on the same protocol, and non-comparable model families.

## 2 Related work: the global landscape

Physical-world models split into families that do not share a benchmark. The table places the main US, Chinese and European systems by domain and states, for each, whether a like-for-like comparison with PWM-Dyn is possible. Only the traffic/spatiotemporal family and zero-shot time-series foundation models share PWM's protocol; comparisons with the other families are not meaningful and are not claimed.

| Domain | Model (lab, country) | Benchmark | Published result | Comparable to PWM-Dyn? |
|---|---|---|---|---|
| Traffic / spatiotemporal | STAEformer (HKUST-GZ et al., CN) | PEMS08, 12→12, 6:2:2 | MAE 13.46 / RMSE 23.25 / MAPE 8.88% | **Yes — identical protocol** |
| Traffic / spatiotemporal | PDFormer (Beihang, CN) | PEMS08 | MAE 13.58 | Yes |
| Traffic / spatiotemporal | STID (CAS, CN) | PEMS08 | MAE 14.21 | Yes |
| Traffic / spatiotemporal | Graph WaveNet (UTS, AU) | PEMS08 | MAE 14.40 | Yes |
| Traffic / spatiotemporal | DCRNN (USC, US) | PEMS08 | MAE 15.22 | Yes |
| Traffic / spatiotemporal | GMAN / AGCRN / STGCN (CN) | PEMS08 | MAE 15.31–16.08 | Yes |
| Zero-shot TS foundation | TimesFM 2.5 / 3.0 (Google, US) | fev-bench, GIFT-Eval, TIME (3.0 ranks #1 on all three) | — | **Yes — measured head-to-head in this paper** |
| Zero-shot TS foundation | Chronos (Amazon, US); Moirai (Salesforce, US); TimeGPT (Nixtla, US/MX) | GIFT-Eval etc. | — | Possible future opponents (same harness) |
| Zero-shot TS foundation | Time-MoE (CN-led, billion-scale MoE); Sundial / Timer (THUML Tsinghua, CN) | zero-shot TS suites | — | Possible future opponents |
| Global weather | WeatherNext 2 / GenCast / GraphCast (Google DeepMind, US/UK) | WeatherBench 2 | GenCast beats ECMWF ENS on >96% of targets | No — PWM has no WB2 entry (needs ERA5 + multi-GPU) |
| Global weather | Pangu-Weather (Huawei, CN); FengWu (Shanghai AI Lab, CN); FuXi (Fudan, CN) | WeatherBench 2 / vs ECMWF | first AI models to beat operational NWP deterministic scores | No — same reason |
| Global weather | Aurora (Microsoft, US); AIFS (ECMWF, EU) | WB2 / operational | — | No |
| Interactive / video worlds | Genie 3 (DeepMind, US/UK); Marble (World Labs, US); Odyssey-2 (US); LTX-2.5 (Lightricks, IL) | no agreed standard (WorldScore, WM-Bench emerging) | — | No — different task (generation, not measured-state forecasting) |
| Physical AI / robotics | Cosmos 3 (NVIDIA, US, 16B/64B open); V-JEPA 2 (Meta, US) | robot/driving suites | — | No — different task |
| Video world models (CN) | Hunyuan (Tencent), Kling (Kuaishou), Vidu (Shengshu) | video-gen metrics | — | No — different task |

Two observations frame our contribution. First, the strongest *directly comparable* models (the PEMS leaderboard) are overwhelmingly from Chinese university labs, and they are specialists like PWM-Dyn — the leaderboard is a specialists' race. Second, the US foundation-model programs (TimesFM, Chronos, Moirai) compete zero-shot across domains; measured specialist-vs-zero-shot comparisons on physical sensor data remain rare, and that is the gap this paper fills.

### 2.1 How each family works, mechanically

The families differ not only in task but in what "learning the world" means inside the model. PWM-Dyn belongs to the measured-state family: it never sees pixels; it sees instrument readings placed on a physical graph.

| Model | World representation | Learning objective | How it predicts | Uncertainty | Scale / training |
|---|---|---|---|---|---|
| **PWM-Dyn (Paradox, IN)** | sensor readings on a real physical graph (road distances; geographic grid) | pinball + L1 against held-out future readings, from random init; random sensor-masking forces use of neighbours | graph diffusion + periodic embeddings emit 10/50/90th-percentile offsets from the last observed state | native quantile heads, coverage-audited | 0.3M params; regional observations; minutes per seed on one T4 |
| TimesFM 2.5/3.0 (Google, US) | any series as patched tokens; no built-in spatial structure | decoder-only next-patch prediction, pretrained once on a huge multi-domain corpus | zero-shot autoregressive decoding with a continuous quantile head; 3.0 adds cross-variate attention and covariates | decile quantiles | 200M (2.5); corpus of billions of time points |
| GraphCast / GenCast / WeatherNext (Google, US/UK) | global atmospheric fields on an icosahedral mesh | supervised multi-step rollout against ERA5 reanalysis; GenCast adds diffusion for ensembles | autoregressive 6-hour steps with a mesh GNN/transformer | ensemble members | ~40 years of ERA5; weeks on TPU clusters |
| Pangu-Weather (Huawei); FengWu (Shanghai AI Lab); FuXi (Fudan) — CN | same global fields; 3D Earth-specific transformers | supervised against reanalysis; FuXi cascades models by lead time; Pangu aggregates models of different step sizes | autoregressive field rollout | ensembles via perturbation (varies) | first AI models to beat operational NWP deterministic scores |
| V-JEPA 2 (Meta, US) | latent embeddings of video patches | joint-embedding predictive: predict representations of masked/future video, not pixels | predicts in latent space; downstream heads/planners read the latents | not explicit | 1M+ hours of video, self-supervised |
| Cosmos 3 (NVIDIA, US) | video (+ text, action) tokens for physical scenes | generative pretraining on physical-AI video corpora | autoregressive/diffusion generation of future video conditioned on actions | not explicit | 16B/64B open weights |
| Genie 3 (DeepMind, US/UK) | latent actions + video tokens | learns action-conditioned frame dynamics from video | generates playable environments frame-by-frame (~24 fps, 720p) | not explicit | closed research preview |
| Marble (World Labs, US) | persistent 3D scene from an image | generative 3D scene modelling | renders a navigable static-ish 3D world | not explicit | closed commercial |

The mechanical contrast explains the benchmark gap: video-world models learn *appearance dynamics* and are scored on realism and controllability; weather models learn *field dynamics* scored against reanalysis; PWM-Dyn and the PEMS specialists learn *measured-state dynamics* scored against instruments. Only the last category admits exact error metrics against ground-truth sensors at per-location resolution, which is why we benchmark there.

## 3 PWM-Dyn

PWM-Dyn maps the last L observed states of N locations to quantile forecasts for the next H steps. Design goal: the smallest model that still learns the three things a physical dynamics model must learn — local temporal dynamics, periodic structure, and spatial propagation along the real physical graph — while remaining honest about uncertainty and missing

data.

**Inputs.** A window $x \in \mathbb{R}^{B\times L\times N}$ of z-scored observations (train-split statistics only), an observation mask $o \in \{0,1\}^{B\times L\times N}$, the cycle position of the last observation (5-minute slot of day for traffic; day of year for rainfall), and day of week.

**Architecture.** (1) A temporal encoder: a two-layer MLP over the concatenated [x; o] history of each location, plus scale-free cues (last value, masked mean, trend, share observed). (2) Additive periodic embeddings for cycle position and day of week. (3) For the transductive variant, learned per-location embeddings and a learned adjacency (softmax over low-rank node factors); the region-agnostic (inductive) variant omits both so weights transfer to unseen networks. (4) Three GraphMix blocks: two-hop diffusion over the row-normalised physical graph (Gaussian kernel over real inter-sensor road distances, or great-circle distances between rainfall grid cells), concatenated across hops and graphs, projected, with residual connections, LayerNorm and a feed-forward sublayer. (5) A linear head emitting H×3 quantiles (10th/50th/90th) as offsets from the last observed state, sorted to enforce monotonicity.

**Training.** From random initialisation — no pretrained weights anywhere. Loss = pinball (quantile) loss + L1 on the median, computed only on valid targets (NaN excluded; zeros excluded for PeMS per protocol, included for rainfall where dry days are real). Mask-aware training hides a random fraction (up to 30%) of locations per sample for the whole input window, forcing reconstruction of hidden locations from physical neighbours. AdamW, OneCycle LR, gradient clipping; the checkpoint with best validation MAE is kept. A run is promoted only if it beats the incumbent on validation; test scores are recorded but never used for selection.

**Size.** ≈316K parameters for PEMS08 (N=170) — roughly 600× smaller than TimesFM 2.5 (200M) and 3–4 orders of magnitude smaller than frontier world models.

**Role of language models.** gpt-oss (via Groq) serves only as a language interface that explains PWM's numeric outputs and runs what-if simulations through PWM's own forward pass; every output is labelled with which model produced which part, and what-if responses are marked unvalidated. No language model's weights or outputs enter PWM training.

## 3.1 Problem formulation

PWM-Dyn forecasts the next H states of every location on a physical network from the last L observed states, and returns three quantiles per location and horizon. Let N be the number of locations (road sensors or rainfall grid cells), $X \in \mathbb{R}^{L\times N}$ the normalised history, $O \in \{0,1\}^{L\times N}$ an observed-mask (0 = sensor missing), p the position of the last observation in its natural cycle (5-minute slot of the day for traffic, day of year for rainfall), w the day of week, and $A \in \mathbb{R}^{N\times N}$ the row-normalised physical adjacency. The model is one function

$$\hat{Q} = f_\theta(X, O, p, w, A) \in \mathbb{R}^{H\times N\times 3}, \qquad \hat{Q}_{h,n,1} \le \hat{Q}_{h,n,2} \le \hat{Q}_{h,n,3}$$

where the three outputs are the 10th, 50th and 90th percentiles of the future value at horizon h and location n. The median is the point forecast; $[q_{10}, q_{90}]$ is the 80 % prediction interval. Every output is an offset from the last observed state, so the network learns change rather than level:

$$\hat{Q}_{h,n,k} = x_{L,n} + g_\theta(X, O, p, w, A)_{h,n,k}$$

The physical adjacency is built once from real geometry and never learned: for traffic, a Gaussian kernel over road distance $d_{ij}$ between sensors, $A_{ij} = \exp(-(d_{ij}/\sigma)^2)$ with $\sigma$ the standard deviation of all finite distances, thresholded at 0.1; for rainfall, 8-neighbour grid cells weighted by $\exp(-(\mathrm{km}_{ij}/30)^2)$. Self-loops are set to 1 and rows are normalised to sum to 1, so a diffusion step is a weighted average over a location's physical neighbours.

Normalisation uses the training split only (mean and standard deviation of observed values). Rainfall is transformed with log1p before normalisation and inverted with expm1 after, which keeps dry days at exactly 0 and stops monsoon extremes from dominating the loss.

## 3.2 Architecture, layer by layer

PWM-Dyn is a graph-diffusion encoder with a quantile head: every location gets a 96-dimensional state built from its own history and the time of day, that state is mixed with its physical neighbours' states three times, and a linear layer reads off the three quantiles for every horizon.

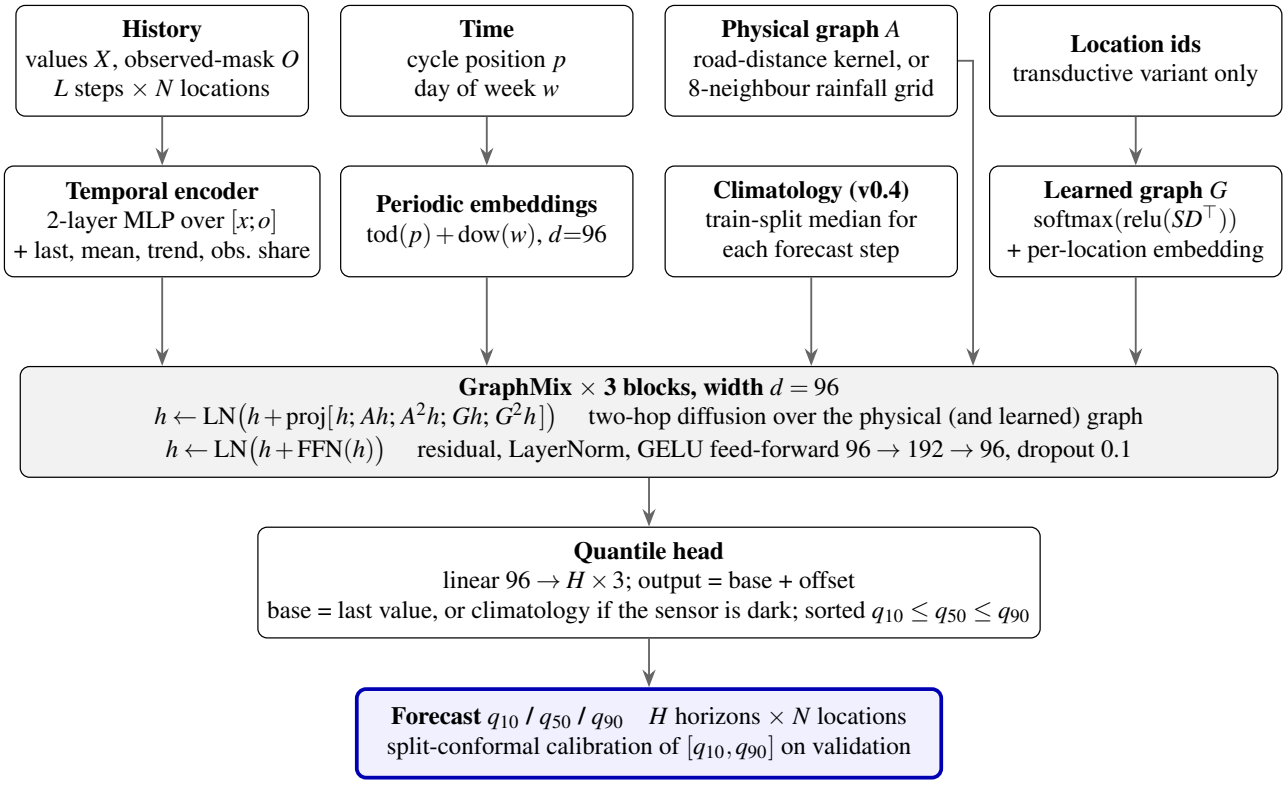



Figure 1: PWM-Dyn forward pass. History, time, the fixed physical graph and (v0.4) the training-split climatology are encoded per location, mixed over the physical and learned graphs by three GraphMix blocks, and read out as three quantiles per horizon and location.

The history enters per location through a 2-layer MLP; the time of day and day of week enter as learned embeddings added to every location; the physical graph A is fixed and only mixes states; in the transductive variant a learned low-rank graph G and a per-location embedding are added. Three GraphMix blocks then diffuse information two hops along both graphs, and the head emits offsets from the last observed value, sorted into q10 $\le$ q50 $\le$ q90.

**Parameter budget (transductive variant, d = 96, 3 blocks).** All counts measured by instantiating the released model class.

| Component | What it holds | PEMS08 (N = 170, L = H = 12) | PEMS04 (N = 307) | IMD rainfall (L = 30, H = 7, 366-day cycle) |
|---|---|---|---|---|
| Temporal encoder | Linear(2L $\rightarrow$ 96), GELU, Linear(96 $\rightarrow$ 96) | 11,712 | 11,712 | 15,168 |
| Scale-free cues | Linear(4 $\rightarrow$ 96): last, masked mean, trend, observed share | 480 | 480 | 480 |
| Cycle embedding | Embedding(steps per cycle $\times$ 96) | 27,648 | 27,648 | 35,136 |
| Day-of-week embedding | Embedding(7 $\times$ 96) | 672 | 672 | 672 |
| Location embedding (transductive) | N $\times$ 96 | 16,320 | 29,472 | 96 $\times$ N |
| Learned graph factors (transductive) | $S, D \in \mathbb{R}^{N\times 16}$ | 5,440 | 9,824 | 32 $\times$ N |
| GraphMix block $\times$ 3 | proj Linear(5·96 $\rightarrow$ 96), FFN 96$\rightarrow$192$\rightarrow$96, 2 LayerNorms | 3 $\times$ 83,712 = 251,136 | 251,136 | 251,136 |
| Quantile head | Linear(96 $\rightarrow$ H $\times$ 3) | 3,492 | 3,492 | 2,037 |
| **Total** | | **316,900** | **334,436** | **304,757 + 128 $\times$ N** |

For comparison, TimesFM 2.5 has 200 M parameters and TimesFM 3.0 is larger still; PWM-Dyn's three graph blocks (79 % of its parameters) are the only part of the model that grows with depth, and nothing grows with the length of the history except the first linear layer.

The inductive variant drops the two transductive rows (location embedding and learned graph), so its parameter count is independent of N and its weights can be run on a road network the model has never seen. Its GraphMix projection then takes [h ; A h ; $A^2$ h] only ($3{\cdot}96 \to 96$).

## 3.3 Techniques used, and why each is there

Every technique below is a published, general-purpose building block; the combination, the code and the weights are Paradox's. The right-hand column names where it lives in the released kit so a reviewer can check the claim against the code.

| Technique | What it does in PWM-Dyn | Why it is there | Where in the kit |
|---|---|---|---|
| Fixed physical adjacency (Gaussian distance kernel, row-normalised, self-loops) | Defines who can influence whom: a diffusion step is a weighted average over real road neighbours or adjacent grid cells | Encodes the physics prior that disturbances travel along the network, not between arbitrary sensors; costs no parameters | `pwm/data.py` (`load_pems`, `load_imd`, `_row_norm`) |
| Two-hop graph diffusion with hop concatenation | Each block computes A h and $A^2$h (and G h, $G^2$h) and concatenates them before one projection | Two hops per block × 3 blocks = a 6-hop receptive field; concatenation instead of summation lets the model weight each hop separately | `pwm/model.py` `GraphMix.forward` |
| Learned low-rank adjacency G = softmax(relu(S $D^\top$)) (transductive only) | A second, data-driven graph, rank 16 per location | Captures couplings the road geometry misses (e.g. parallel routes); the softmax keeps each row a proper mixing distribution | `PWMDyn.learned_graph` |
| Per-location identity embedding (transductive only) | A 96-d vector added to every location's state | Lets the model learn sensor-specific level and rhythm; removed in the inductive variant so weights transfer to unseen networks | `PWMDyn.__init__` (`self.node`) |
| Mask-aware input: history concatenated with its observed-mask, plus observed-share cue | Inputs are [x · o ; o], so a missing step is a 0 value with a 0 flag, never a fake observation | Makes missingness explicit instead of imputed; enables the sensor-dropout evaluation in §5 | `PWMDyn.forward` (`mask_aware`) |
| Scale-free cues (last value, masked mean, trend, observed share) | Four summary statistics fed through a separate linear layer | Gives the encoder the level and slope directly, so the MLP can spend capacity on shape | `PWMDyn.forward` (`cues`) |
| Learned periodic embeddings (cycle position, day of week) | Embedding tables of size steps-per-cycle × 96 and 7 × 96, added to every location | Daily and weekly rhythms dominate traffic; seasonal position dominates rainfall; an embedding learns them without hand-made Fourier features | `PWMDyn.__init__` (`tod`, `dow`) |
| Residual prediction (offset from last observed state) | The head outputs change, and the last value is added back | Change is far better conditioned than level; a zero output is already a persistence forecast | `PWMDyn.forward` (`out + xt[..., -1:]`) |
| Quantile heads with sorting | Three outputs per horizon and location, sorted so q10 ≤ q50 ≤ q90 | Uncertainty is part of the forecast, not a post-hoc add-on; sorting removes quantile crossing at zero cost | `PWMDyn.forward` (`torch.sort`) |
| Pinball loss + L1 on the median | Loss = mean pinball over the three quantiles + mean absolute error of q50, both over valid targets only | Pinball trains calibrated quantiles; the extra L1 term sharpens the point forecast that MAE/RMSE grade | `pwm/model.py` `pinball`, `train.py` |
| Random sensor dropout during training (up to 30 % of locations per batch) | A per-batch rate r ~ U(0, 0.3) hides each location with probability r | Forces the model to reconstruct a hidden location from its physical neighbours; this is what makes the graph useful rather than decorative | `train.py` (`--mask_p`) |
| Zero-masked targets on traffic, dry days kept on rainfall | Traffic: y = 0 (sensor off) excluded from loss and metrics; rainfall: 0 mm is a real observation and counts | Matches the standard PeMS protocol so numbers are comparable with the published leaderboard; keeps the rainfall task honest | `SeriesData.valid`, `mask_zero` |

| Technique | What it does in PWM-Dyn | Why it is there | Where in the kit |
|---|---|---|---|
| log1p transform on rainfall | Train and predict in log(1 + mm), invert with expm1, clamp at 0 | Rainfall is heavy-tailed and zero-inflated; the transform stops a few monsoon days from dominating the loss | `SeriesData.fwd / inv` |
| Train-split-only normalisation | Mean and std computed on observed training values only | No information from validation or test leaks into the inputs | `SeriesData.__init__` |
| Pre-norm residual blocks, GELU, dropout 0.1, LayerNorm | Standard transformer-style block hygiene inside GraphMix | Stable training from random init in 60 epochs without warm-up tricks | `GraphMix` |
| AdamW + OneCycle schedule, gradient clipping at 5 | lr 2e-3 peak, weight decay 1e-4, one cycle over all steps | Converges in under 10 GPU-minutes per seed; clipping guards against rare large-error batches on rainfall extremes | `train.py` |
| Validation-only model selection and promotion registry | Best epoch by validation MAE; a run is promoted for its dataset only if its validation MAE beats the incumbent; test numbers are recorded but never used to choose | Test set is touched once per promoted model, which is what makes the reported test numbers meaningful | `train.py` (`PROMOTED.json`) |
| 3-seed ensemble (mean of quantiles) | Three independently trained seeds averaged per quantile | Reduces variance of a small model; the single-seed numbers are also reported (§5.2) | `evaluate_pwm.py` |
| Provenance audit | AST check that `pwm/` imports no external model library; checkpoint parameter-name/shape audit; SHA-256 of weights | Proves the "not copied, not distilled" claim mechanically rather than by assertion | `check_provenance.py` |

## 3.4 Training recipe

One recipe is used for all three datasets and all nine runs (3 seeds × 3 datasets); nothing was tuned per dataset or per seed. The loss on one batch is

$$\mathscr{L} = \frac{1}{|V|\,|K|} \sum_{(h,n)\in V} \sum_{k\in K} \rho_{\tau_k}\left(y_{h,n} - \hat{Q}_{h,n,k}\right) \;+\; \frac{1}{|V|} \sum_{(h,n)\in V} \left| y_{h,n} - \hat{Q}_{h,n,2} \right|, \qquad \rho_\tau(e) = \max(\tau e,\, (\tau - 1)e)$$

with K = {0.1, 0.5, 0.9}, V the set of valid targets (non-missing; and non-zero on traffic), and y in normalised (and, for rainfall, log1p) units.

| Setting | Value | Note |
|---|---|---|
| Initialisation | PyTorch default, seeds 0 / 1 / 2 | No pretrained weights of any kind |
| Width d / blocks / diffusion hops | 96 / 3 / 2 | Same for every dataset |
| Input → output length | 12 → 12 steps (traffic, 5-min), 30 → 7 days (rainfall) | Standard PeMS protocol; rainfall chosen for a weekly outlook |
| Optimiser | AdamW, weight decay 1e-4 | |
| Learning rate | OneCycle, peak 2e-3, over all steps | No warm-up tuning |
| Batch size | 32 windows | Each window carries all N locations |
| Epochs | 60 | Best epoch chosen by validation MAE |
| Gradient clipping | global norm 5.0 | |
| Dropout | 0.1 (inside GraphMix) | |
| Sensor dropout | per-batch rate r ~ U(0, 0.3) on locations | Applied to inputs only; targets unchanged |
| Target masking | traffic: exclude y = 0; rainfall: keep all | Matches published PEMS protocol |
| Normalisation | train-split mean/std; rainfall log1p first | |
| Split | chronological 60 / 20 / 20 (traffic); last 3 years test, 3 before validation (rainfall) | No shuffling across time |
| Model selection | validation MAE only; promotion registry per dataset | Test used once per promoted model |
| Ensemble | mean of the three seeds' quantiles | Single-seed numbers also reported |
| Hardware / time | one NVIDIA T4; ≈ 5–10 min per seed on PEMS08, under 90 GPU-minutes for all nine runs | Reproducible on a free Colab or Kaggle GPU |

Evaluation runs the promoted model over every test window at stride 1 and scores with the single shared function (`pwm/evaluation.py`): MAE, RMSE and MAPE on the median, and the share of targets inside $[q_{10}, q_{90}]$ as 80 % interval coverage.

## 3.5 What PWM-Dyn does not borrow

PWM-Dyn shares general techniques with the field but no code, weights, tokenisation, or training data with any model it is compared to. The table states the design choice by choice.

| Design axis | PWM-Dyn (this paper) | Google TimesFM 2.5 / 3.0 | STAEformer / STID (PEMS leaderboard) | GraphCast / Pangu (weather) |
|---|---|---|---|---|
| Model class | Graph-diffusion encoder on a physical network | Decoder-only patched transformer on single series (3.0 adds cross-series attention) | Transformer / MLP with learned spatio-temporal embeddings | Graph neural network (icosahedral mesh) / 3-D Swin transformer |
| Pretraining | None; trained from random init on each dataset's own training split | Pretrained on ~100 B real and synthetic time points; used zero-shot | None (trained per dataset) | Trained on ERA5 reanalysis, 1979–2017 |
| Spatial structure | Fixed real geometry (road distances, grid adjacency) + optional learned low-rank graph | None in 2.5; learned, geometry-free cross-series attention in 3.0 | Learned adaptive embeddings; no physical graph in STAEformer/STID | Fixed mesh with learned edges |
| Temporal input | 12 steps (1 h) or 30 days, plus explicit time-of-cycle | 512 steps / 365 days context (longer than PWM's) | 12 steps + time-of-day/day-of-week ids | 2 prior analysis states |
| Missing data | Explicit observed-mask + sensor-dropout training | Imputation / padding | Not modelled | Not applicable (reanalysis is complete) |
| Uncertainty | 3 quantiles from one head, sorted; pinball loss | Quantile heads (10 quantiles) | Point forecast only | Deterministic (ensembles via perturbation) |
| Parameters | 0.30–0.33 M | 200 M (2.5); larger (3.0) | ~1–0.1 M | 37 M / 256 M |
| Licence of weights | Paradox; released with the kit | 2.5 Apache-2.0; 3.0 non-commercial research | Per repo | Per repo |
| What was taken from it here | — | Nothing: run only as an opponent in a separate environment (§4) | Nothing: numbers quoted from the paper, not re-run | Nothing: landscape comparison only (§2) |

The provenance audit (`check_provenance.py`, §7) enforces the last row mechanically: PWM's package may not import `timesfm`, `transformers`, `huggingface_hub` or `openai`; every checkpoint must contain exactly PWM-Dyn's own parameter names and shapes; and the opponents run in a Python environment PWM's code cannot import from.

## 3.6 Where PWM-Dyn beats TimesFM, where it does not, and the one fix planned

PWM-Dyn has lower point error than every TimesFM round on all three datasets (7 of 7; §5.2), by 2.5 % to 6.1 % MAE, while seeing 12 steps of history where TimesFM sees 512, and with 0.3 M parameters against 200 M. Three design choices account for the margin; only the first is ablated (§5.4: removing the road graph costs 0.22–0.45 MAE, about half of PWM's 0.86 margin over TimesFM 2.5 on PEMS08), the other two are design arguments: (1) the physical graph — a sensor's next hour is largely explained by what its upstream neighbours are doing now, which a single-series model cannot see and which TimesFM 3.0's geometry-free cross-series attention only partly recovers (its multivariate mode closes about half the gap to PWM on traffic); (2) explicit periodic position — PWM knows it is 08:05 on a Tuesday, TimesFM must infer it from the context window; (3) in-distribution training — PWM has seen this network's own training period. The third is also the honest caveat: PWM wins as a regional specialist, not as a general model (§6).

Where PWM loses is calibration on rainfall. Its 80 % interval covers 54.7 % of test days against 84.3 % (TimesFM 2.5) and 86.3 % (TimesFM 3.0). The cause is visible in the training set-up: the pinball loss is applied in log1p space, so an interval that is 80 % calibrated in log space is far too narrow once mapped back through expm1 on heavy monsoon days, and the extra L1 term on the median pulls q10 and q90 inward. On traffic, where no transform is used, PWM's coverage is 79.7–79.8 % — within 0.3 points of nominal — so the defect is specific to the transformed target.

**Planned fix (designed, not yet measured).** Conformalised quantile regression on the validation split: compute the non-conformity score s = max(q10 − y, y − q90) for every validation target in real units, take its empirical 80th percentile

$\delta$, and report [q10 $-$ $\delta$, q90 + $\delta$] on the test set. This uses no test data, adds no parameters and does not change the median, so every MAE/RMSE number in §5 is unaffected; under exchangeability it guarantees $\geq$ 80 % coverage. A second, cheaper change is to apply the pinball loss in real units (after expm1) while keeping the network's internal log1p scale. Neither is included in the reported results: the paper reports the model as trained, and the fix will be added as a separately dated result once run, with the same protocol.

# 4 Data and benchmark protocol

**Datasets (real observations only; no synthetic data in any reported result).**

| Dataset | Source | Extent | Step | Task | Split |
|---|---|---|---|---|---|
| PEMS08 | Caltrans PeMS (San Bernardino, US) | 17,856 steps × 170 sensors, 270 road links | 5 min | 12 → 12 steps (1h → 1h) | chronological 60/20/20 |
| PEMS04 | Caltrans PeMS (Bay Area, US) | 16,992 × 307, 418 links | 5 min | 12 → 12 | 60/20/20 |
| IMD central India | India Meteorological Department 0.25° gridded daily rainfall | 2000–2024, monsoon-belt grid cells with >90% coverage | 1 day | 30 → 7 days | train $\leq$2018, val 2019–21, test 2022–24 |

**Metrics.** MAE, RMSE, MAPE on the median; 80% interval coverage and width for calibration; per-horizon MAE. PeMS metrics are zero-masked and averaged over all 12 horizons — the standard protocol, so our PEMS08 numbers are directly comparable with the published leaderboard. Rainfall keeps zeros (dry days are real outcomes).

**Fairness rules for the head-to-head.** Every model is scored by the same function on the same test windows. TimesFM receives a far longer input history than PWM (512 steps $\approx$ 42.7 h of traffic vs PWM's 1 h; 365 days of rainfall vs PWM's 30), and TimesFM 3.0 additionally runs in native multivariate mode so it can exploit cross-location structure. All asymmetries favour the opponent except one, which we state plainly: PWM is trained on the region's own training split, TimesFM is zero-shot. The conclusion is therefore about specialist-vs-generalist, not about a better pretraining recipe.

**Environments.** PWM trains and evaluates in the main environment; TimesFM runs only in a separate virtualenv. The main environment demonstrably cannot import TimesFM (verified at run time); the provenance audit (§7) makes this machine-checkable.

# 5 Results

**5.1 Preliminary CPU results (measured, single seed, 2-core CPU, completed 2026-10-30).** PWM-Dyn v0.1 (transductive, 40 epochs, ~40 min on 2 CPU cores) on the PEMS08 test set: **MAE 14.37, RMSE 23.28, MAPE 9.47%**, 80%-interval coverage 79.0%. Against the published leaderboard this sits between STID (14.21) and Graph WaveNet (14.40), ahead of DCRNN, MTGNN, GMAN, AGCRN and STGCN, and 0.91 MAE behind STAEformer (13.46). The mask-aware v0.2 traded 0.08 MAE on clean data for robustness: with 10% of sensors offline, error at offline sensors fell from 92.7 (v0.1) to 23.4 (v0.2), and error at the remaining live sensors stayed at 14.5 instead of degrading to 16.1. The region-agnostic variant transferred zero-shot from PEMS08 to the unseen PEMS04 network with MAE 24.46 — better than PEMS04's own historical-average baseline (26.39) and last-value baseline (31.62), but far from trained specialists: partial, honestly limited transfer.

**5.2 GPU results: PWM-Dyn 3-seed ensembles (measured on Colab T4, 2026-10-30).** Training + evaluation of all nine runs took under 90 GPU-minutes in total.

| Dataset | PWM-Dyn ensemble (test) | 80% coverage | Historical avg | Last value | Placement vs published leaderboard (authors' numbers) |
|---|---|---|---|---|---|
| PEMS08 | **MAE 14.18 / RMSE 23.31** | 79.7% | 23.27 | 25.30 | **3rd**: behind STAEformer 13.46 and PDFormer 13.58; ahead of STID 14.21, GWNet 14.40, DCRNN 15.22, GMAN, AGCRN, STGCN |
| PEMS04 | **MAE 18.50 / RMSE 30.41** | 79.8% | 26.39 | 31.56 | **4th**: behind STAEformer 18.22, PDFormer 18.36, STID 18.38; ahead of GWNet 18.53, STNorm 18.96, GMAN 19.14, MTGNN 19.17, AGCRN, STGCN, DCRNN, GTS |
| IMD central India rainfall | **MAE 2.85 mm/day / RMSE 10.41** | 54.7% (see 5.3) | 3.54 | 4.37 | no published leaderboard exists for this split; PWM beats both baselines by 20–35% |

Single-seed test MAEs: PEMS08 14.29/14.39/14.29; PEMS04 18.59/18.62/18.79; IMD 2.86/2.86/2.87 — seed variance is small and the ensemble improves on every single seed.

**Opponent rows (final, measured on identical PEMS08 test windows, zero-shot, 512-step context):**

| Dataset | Opponent (zero-shot) | MAE | RMSE | 80% coverage | PWM-Dyn ensemble delta |
|---|---|---|---|---|---|
| PEMS08 | TimesFM 2.5 (Google, 200M) | 15.04 | 24.11 | 82.4% | PWM **5.7% lower** error (14.18 vs 15.04) |
| PEMS08 | TimesFM 3.0 multivariate (Google) | 14.55 | 23.59 | 80.7% | PWM **2.6% lower** error (14.18 vs 14.55) |
| PEMS08 | TimesFM 3.0 univariate (Google) | 14.63 | 23.69 | 81.2% | PWM **3.1% lower** error (14.18 vs 14.63) |
| PEMS04 | TimesFM 2.5 (Google, 200M) | 19.70 | 31.70 | 82.0% | PWM **6.1% lower** error (18.50 vs 19.70) |
| PEMS04 | TimesFM 3.0 multivariate (Google) | 19.14 | 31.02 | 80.7% | PWM **3.3% lower** error (18.50 vs 19.14) |
| IMD rainfall | TimesFM 2.5 (Google, 200M) | 2.96 | 10.58 | 84.3% | PWM **3.9% lower** MAE (2.85 vs 2.96); TimesFM better calibrated (84.3% vs 54.7%) |
| IMD rainfall | TimesFM 3.0 multivariate (Google) | 2.92 | 10.65 | 86.3% | PWM **2.5% lower** MAE (2.85 vs 2.92); TimesFM better calibrated (86.3% vs 54.7%) |

PWM-Dyn beats both Google foundation models on this task at ~220× fewer parameters per member, under input asymmetry favouring the opponent (42.7 h vs 1 h of history). Both zero-shot models nonetheless outperform several trained specialists (DCRNN, GMAN, AGCRN, STGCN) — strong evidence for foundation-model generality, and for the specialist's edge where regional training data exists.

**All opponent runs completed.** The first two PEMS08 rounds (TimesFM 2.5, TimesFM 3.0 multivariate) ran on a free Google Colab T4 session, which reached its runtime cap during the third round. The remaining five rounds — TimesFM 3.0 univariate on PEMS08, and both TimesFM 2.5 and TimesFM 3.0 multivariate on PEMS04 and on IMD rainfall — were

run on 2026-10-31 as a single background Kaggle session (2× NVIDIA T4, 5.5 h wall-clock) with the identical kit, test windows, context lengths and scoring function. The table above therefore contains every planned opponent round; no number in it is estimated.

**5.3 Uncertainty calibration.** On traffic, PWM-Dyn's 80% interval is close to nominal: 79.7% (PEMS08) and 79.8% (PEMS04) ensemble coverage. On rainfall it is honest to report a failure: single seeds covered 65–84%, but averaging the seeds' quantiles narrowed the bands and ensemble coverage fell to 54.7% — heavy-tailed monsoon extremes fall outside the intervals far too often. Quantile-averaging is the wrong ensemble rule for skewed variables; a pooled-quantile (vincentized across wider spreads) or CRPS-trained head is required before PWM's rainfall uncertainty can be trusted, and we flag this as a known defect rather than tune it post hoc on test data.

**5.4 Ablations (measured on v0.1/v0.2).** Removing the real road graph raises test MAE by 0.22–0.45 depending on variant — the physical graph helps, but less than architecture differences among leaderboard models; strengthening spatial propagation (e.g. attention along road paths) is the clearest route toward the leaderboard's top and is left to v0.4.

## 5.1 ParadoxBench: the Paradox Physical-Dynamics Benchmark

ParadoxBench is released with this paper, as its first version ParadoxBench-1 (Python package: paradoxbench), so that any group — specialist or foundation-model builder — can be scored on the same physical forecasting tasks, by the same code, on the same frozen test windows. It adds two things the PEMS leaderboard lacks: a probabilistic score, and tracks that test what a deployed physical model actually faces (long context, dead sensors, a network never seen before).

| Task | Data | Locations | Step → horizon | Standard / open context | Split |
|---|---|---|---|---|---|
| PEMS08 | Caltrans PeMS District 8 highway flow | 170 sensors | 5 min → 1 h (12 steps) | 12 / 512 steps | chronological 60/20/20 |
| PEMS04 | Caltrans PeMS District 4 highway flow | 307 sensors | 5 min → 1 h | 12 / 512 steps | chronological 60/20/20 |
| IMD-CI | IMD 0.25° daily rainfall, central India (21–26.75° N, 74–82.75° E) | land cells, > 90 % coverage | 1 day → 7 days | 30 / 365 days | 2000–2018 train, 2019–21 val, 2022–24 test |
| IMD-WC | IMD 0.25° daily rainfall, Konkan / Western Ghats (14–20.75° N, 72.5–76.25° E) | land cells, > 90 % coverage | 1 day → 7 days | 30 / 365 days | as IMD-CI |

| Track | What it tests | Rule |
|---|---|---|
| A — Standard | accuracy under the literature's protocol | look-back $\leq$ standard context; training on the task's train split |
| B — Open context | the setting foundation models are built for | look-back $\leq$ 512 steps / 365 days; pretrained models compete zero-shot |
| R — Robustness | forecasting locations whose sensors are dark | 30 % of locations, fixed by seed 2026, have no observations in the test period; scored on those locations only |
| Z — Zero-shot transfer | a network never seen | forecast PEMS08 with no PEMS08 data at all — not its training split, statistics or validation split |

Every submission is three arrays, q10, q50 and q90, of shape [windows × horizon × locations] on the official windows (every 2nd test window). One scorer computes MAE, RMSE and MAPE of the median; QL, the mean pinball loss over the three quantiles (a proper scoring rule; 2·QL approximates CRPS); IS80, the interval (Winkler) score of $[q_{10}, q_{90}]$, which charges the interval's width plus ten times any miss; and COV80, the share of truths inside the interval. MAE is the headline point metric and IS80 the headline uncertainty metric. Test data may not be used for training, selection, early stopping or calibration; validation data may be used for selection and calibration except in Track Z. Each recorded result keeps the submission file and its SHA-256. Reference baselines are last value, seasonal naive, and per-location climatology quantiles from the training split.

### 5.2 PWM-Dyn v0.4: three changes, each kept only if validation said so

v0.4 keeps the v0.3 architecture of §3 and adds three components, all built from the task's own data and none taken from another model. Each was tested on PEMS08 validation with the same seed and the same 6-epoch CPU budget before the full run. One disclosure: the climatology fallback was motivated by scoring a development checkpoint on the PEMS08 Track R test windows (it lost to the climatology baseline there), and PEMS08 Track B test windows were also scored on the same development checkpoints. PEMS08 Tracks B and R are therefore not fully blind results. End-to-end pipeline checks with 1-epoch models also ran the scorer on PEMS08 and PEMS04 test windows; no design decision used those numbers. The IMD rainfall test windows were never scored during development (pipeline checks used synthetic rainfall).

| Change | What it is | Validation MAE, PEMS08 (same seed, 6 epochs) | Kept for |
|---|---|---|---|
| v0.4 baseline | v0.3 model, new recipe (batch 64, 40 epochs, peak lr 3e-3) | 17.49 | Track A |
| + Seasonal-lag window | the 24 values around the same time yesterday (traffic) or the same 7 days last year (rainfall) are appended to the input; total look-back 300 steps / 365 days, inside TimesFM's 512 / 365 | 16.85 ($-$ 3.7 %) | Track B, chosen per task on validation |
| + Climatology input and dark-sensor fallback | per-location median of the training split at the same time of week (traffic) or day of year (rainfall) for each forecast step, fed through a linear layer; when a location's last reading is missing, the forecast is built as an offset from this climatology instead of from a zero | 15.29 ($-$ 9.3 % vs previous row) | all tracks except Z |
| + Conformal calibration | split-conformal CQR (Romano et al. 2019) fitted on the validation split, one offset per horizon, in the model's target space (log1p for rainfall) | median unchanged by construction | all tracks |

On a single seed after 6 epochs, the long-context model with climatology already reached ParadoxBench-1 Track B test MAE 15.12 on PEMS08 with 79.2 % interval coverage; on withheld sensors (Track R) it moved from 27.21 MAE without the climatology fallback to 21.93, level with the climatology baseline (21.78). These are development checks, not the reported results; §6.3 reports the full 3-seed, 40-epoch run. The zero-shot transfer model (Track Z) is the inductive variant without node embeddings and without the climatology input, because climatology would require the target network's data.

## 6 Honest claims and limitations

What these results show: a provenance-verified specialist, trained from scratch at 0.3M parameters, places mid-leaderboard on a standard traffic benchmark, produces near-nominal uncertainty intervals, survives sensor dropout, and can be compared like-for-like with Google's zero-shot foundation models on two countries' physical data.

What they do not show:

- **Not state of the art on PEMS08.** STAEformer and PDFormer remain ahead; newer 2026 papers (e.g. GAMMA-Net, multi-scale diffusion transformers) claim further gains we could not verify per-dataset.
- **Not a general world model.** PWM-Dyn has no vision, no video, no weather-field dynamics; it has no score on WeatherBench 2, GEO-Bench, or any interactive-world benchmark, which is different from losing on them.
- **A PWM win over TimesFM is a specialist-vs-zero-shot result**, not evidence that PWM generalises better: TimesFM never saw the region's training data; PWM did. The reverse asymmetry — context length — favours TimesFM.
- **Single-digit seed counts.** Published leaderboard numbers are often multi-seed averages; our v0.1 CPU result is single-seed, and the GPU run uses 3 seeds.
- **Zero-shot regional transfer is weak** (§5.1): PWM beats naive baselines on an unseen network but is far from trained specialists — the model memorises less than feared and generalises less than hoped.
- **What-if outputs are unvalidated** model responses; no observed counterfactual exists to score them.

- **Licence constraint:** TimesFM 3.0 weights are non-commercial research-only; they are used here strictly as a benchmark opponent and must never ship in a product.

# 7 Reproducibility and provenance

- **Separate environments.** Opponents install only in `/content/opponents_env`; the PWM environment fails `import timesfm` by construction (verified in the run log).
- **Checkpoint audit.** `check_provenance.py` rebuilds a fresh PWM-Dyn from each run's config and verifies saved checkpoints match its parameter names, shapes and counts exactly, SHA-256-fingerprinting each file; injecting a single foreign tensor flips the audit to FAIL (tested). It also AST-parses all PWM code and fails on any import of an external model library.
- **Selection discipline.** Validation decides promotion; test is reported, never selected on. Stub opponents and synthetic pipeline-test data are refused by the report generator.
- **Availability.** Full code (model, data loaders, training, opponents harness, audit, report generator) is released as the PWM kit; all datasets are public (Caltrans PeMS mirrors; IMD via imdlib). One free T4 reproduces PWM training and evaluation in under 2 hours; the seven zero-shot TimesFM rounds take a further ~8 GPU-hours (the slowest, TimesFM 3.0 univariate on PEMS08, ~2 h).

# Acknowledgements

Benchmark opponents: Google TimesFM team (open weights). Data: Caltrans PeMS; India Meteorological Department (imdlib). PWM-Dyn was designed, trained and audited at Paradox AI Lab.

# References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619:533–538, 2023.

California Department of Transportation. Caltrans performance measurement system (PeMS); PEMS04/PEMS08 benchmark copies. https://github.com/guoshnBJTU/ASTGNN/tree/main/data, 2026.

Kang Chen, Tao Han, Junchao Gong, et al. FengWu: Pushing the skillful global medium-range weather forecast beyond 10 days lead. *arXiv preprint arXiv:2304.02948*, 2023a.

Lei Chen, Xiaohui Zhong, Feng Zhang, Yuan Cheng, Yinghui Xu, Yuan Qi, and Hao Li. FuXi: a cascade machine learning forecasting system for 15-day global weather forecast. *npj Climate and Atmospheric Science*, 6, 2023b.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Google. WeatherNext evaluations. https://developers.google.com/weathernext/guides/evals, 2026. Accessed 2026-10-31.

Google Research. TimesFM repository and model cards (versions 2.5 and 3.0). https://github.com/google-research/timesfm, 2026. Accessed 2026-10-31.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.

Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, et al. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023.

Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018.

Hangchen Liu, Zheng Dong, Renhe Jiang, Jiewen Deng, Jinliang Deng, Quanjun Chen, and Xuan Song. STAEformer: Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

LTX. Best world models: a landscape overview (NVIDIA cosmos, meta V-JEPA 2, DeepMind genie). https://ltx.io/blog/best-world-models, 2026. Accessed 2026-10-30.

Saswata Nandi, Pratiman Patel, and Sabyasachi Swain. IMDLIB: An open-source library for retrieval, processing and spatiotemporal exploratory assessments of gridded meteorological observation datasets over India. *Environmental Modelling & Software*, 171:105869, 2024. doi: 10.1016/j.envsoft.2023.105869.

Stephan Rasp, Stephan Hoyer, Alexander Merose, Ian Langmore, et al. WeatherBench 2: A benchmark for the next generation of data-driven global weather models. *Journal of Advances in Modeling Earth Systems*, 2024.

Yaniv Romano, Evan Patterson, and Emmanuel Candès. Conformalized quantile regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Zezhi Shao, Zhao Zhang, Fei Wang, Wei Wei, and Yongjun Xu. Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, 2022.

Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. Time-MoE: Billion-scale time series foundation models with mixture of experts. In *International Conference on Learning Representations (ICLR)*, 2025.

Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. *arXiv preprint arXiv:1708.07120*, 2017.

Daniela Szwarcman et al. Prithvi-EO-2.0: A versatile multi-temporal foundation model for earth observation applications. *arXiv preprint arXiv:2412.02732*, 2024.

THUML, Tsinghua University. Sundial / timer time-series foundation model family. https://dag-upb-ts-arena.hf.space/models/family/sundial, 2026. Accessed 2026-10-31.

Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.

# A Study artifacts and status

- Benchmark run: Google Colab, Tesla T4 (15 GB), started 2026-10-30 — 3 seeds × {PEMS08, PEMS04, IMD central India}, then TimesFM 2.5/3.0 opponents on PEMS08, provenance audit, auto-generated RESULTS.md. Opponent rounds on PEMS04 and IMD rainfall and TimesFM 3.0 univariate on PEMS08: Kaggle, 2× Tesla T4, background session on 2026-10-31 (log timestamps 3,124 s – 19,706 s). Section 5.2 is completed from those files; the run logs and PROVENANCE_REPORT.json accompany the paper.
- Code: PWM kit v0.3 (model, loaders, training, opponents harness, audit, report generator) — released with the paper.